bases de données avancées

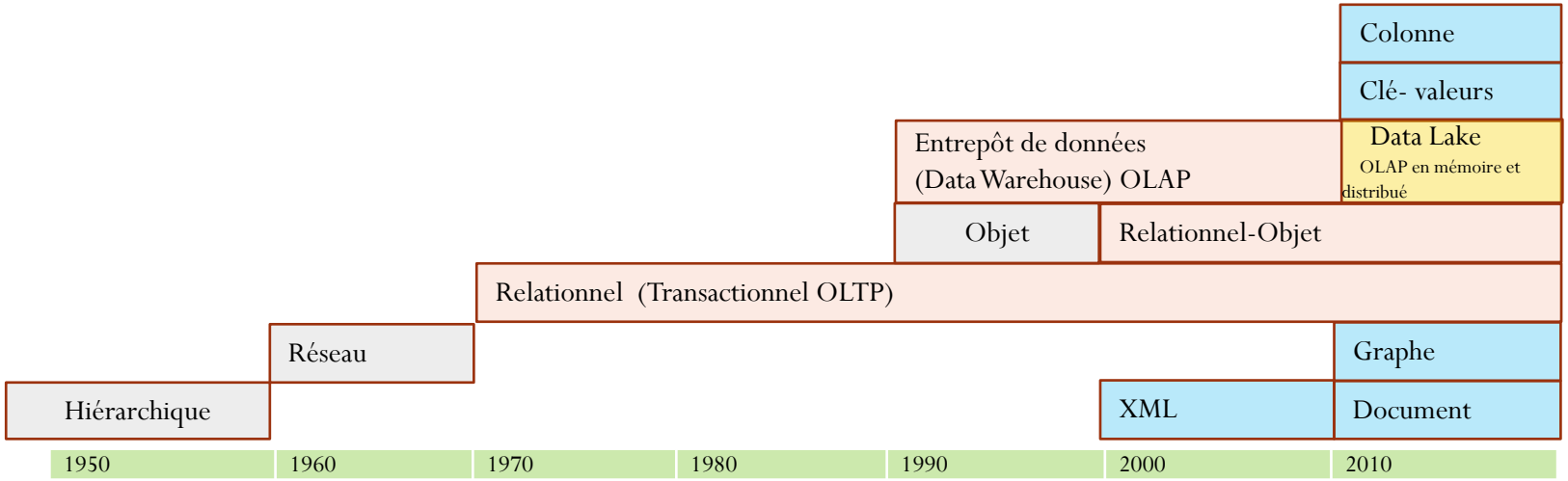
Bases de données Relationnelles Objets

Présenté par:
Amal HALFAOUI (Epse GHERNAOUT)
Amal.halfaoui@univ-tlemcen.dz
amal.halfaoui@gmail.com

SQL3: Partie 1
Modélisation/Création des types

Historique des bases de données

- Evolution des Types SGBD



- Certains modèles n'ont pas réussi à s'imposer : Hiérarchique, réseaux, objets
- Dominance forte du modèle relationnel grâce à un modèle théorique puissant et simple: **schéma**, **normalisation** et **transaction**
- Nouveaux modèles (**NoSQL**) qui ne reposent plus sur le modèle relationnel (retour aux modèles <u>hiérarchique</u> et <u>réseau</u>)

Pourquoi faire évoluer le modèle relationnel

- Le concept de tables est bien adapté pour représenter des types (alphanumériques, temps…)
- Le concept de transaction offre de très bonnes performances
- Le modèle repose sur des bases théoriques fermes
- L'existence d'outils matures
- La pauvreté des données modélisées : pourquoi un <u>champs de la table ne pourrait pas lui-même être une table</u> !!
- La nécessité de prendre en considération des données complexe
- Il n'est pas possible d'intégrer des opérations avec les données.

Le modèle relationnel est mal adapté pour les application multimédias ou géographiques

Les offres de l'OR

- Définir de **nouveaux types complexes** avec des fonctions pour les manipuler
- Une colonne peut contenir une collection (**ensemble, sac, liste**)
- Supporter des données complexe tel que : Données multimédias
- Intégrer certains concepts du modèle objet : **Identité d'objet, Encapsulation, Héritage**
- Ligne considérée comme un objet, avec un identificateur (Object Identifier OID)
- Utilisation de **références** aux objets
- Extensions du langage SQL (SQL3 ou SQL99) pour la recherche et la modification des données

SQL3: Modélisation/Création des types

Les domaines usuels de SQL tels que VarChar, Number, Date…

La possibilité de définir de nouveaux types grâce au constructeur **TYPE** dont il existe plusieurs formes:

1-Type abstrait de donnée (**Object**);

2-Type ensembliste ou collection d'ensemble imbriquée

-Collection ordonnée limité **Varray**

-Collection non ordonnée illimitée **Table**

SQL3: Modélisation/Création des types

Type

Objet TAD

```
CREATE TYPE nom_type ...AS OBJECT ()
```

```
CREATE  TYPE nom_type AS OBJECT
     (defcolonne  REF nom-type)
```

```
CREATE TYPE nom_type;
```

Collection

Tableaux prédimensionnés

```
CREATE nom_type TYPE ...AS
varray ()
```

Tables imbriquées

```
CREATE TYPE   nom-type    AS TABLE OF
nom-type;

CREATE TABLE nom-table of nom-type
NESTED TABLE nom-attribut-de-type-
table-emboîtée STORE AS nom-table-
annexe
```

1 Type Abstrait de Données TAD

1 Type abstrait de données(TAD ou type Object)

Définition

Un type abstrait de données (TAD) est un nouveau type défini par l'utilisateur.

Il peut correspondre à une structure de données partagées ; dans ce cas, il peut être utilisé dans une ou plusieurs tables et entrer dans la composition d'un ou plusieurs autres types.

Syntaxe

```
Create  [or replace] type NomType as object
( nom attr1   TypeAttr1,
  nom attrn   TypeAttrn
 )
```

1 Type abstrait de données(TAD ou type Object)

Exemple

```
CREATE TYPE   t_adresse    AS OBJECT
(  num NUMBER,
   rue VARCHAR(30),
   ville VARCHAR(20),
   codepostal CHAR(5)
) ;
```

On peut ensuite utiliser ce type objet créé(ou type utilisateur)

a) pour définir une table relationnelle standard

b) pour définir une table relationnelle objet

c) pour définir d'autres types objets qui contiennent cette structure.

A- Utilisation du type dans la définition d'un autre type

```
CREATE TYPE t_personne AS OBJECT
( nom VARCHAR(30),
  prenom VARCHAR(30),
  adresse  t_adresse
) ;
```

B- Utilisation du type dans une table relationnelle

```
CREATE TABLE employes
(
num NUMBER,
dept NUMBER,
salaire NUMBER,
adresse t_adresse,    /%type objet
nom VARCHAR(30),
PRIMARY KEY (num)    /%on peut définir des contraintes
habituelles sur la table
) ;
```

C- Utilisation du type pour créer une table objet

- Une table objet relationnelle est une table qui contient des éléments de type objet.
- Chaque élément est identifié par un numéro appelé OID (Object Identifier).OID (Object Identifier).

```
Create type TVoiture as object
( modele varchar2(15),
  annee date);

Create table Voiture of Tvoiture;
```

Remarque

On peut écrire des contraintes comme en relationnel standard lors de la création de la table, on peut définir des clés, etc.

CREATE TABLE Voiture OF Tvoiture (modele Primary key) ;

1 TAD Exemple

```
CREATE TYPE adresse_objtyp AS OBJECT (
num  NUMBER(4),
rue   VARCHAR2(20),
ville  VARCHAR2(20),
codePostal NUMBER(5)
);

CREATE OR REPLACE TYPE client_objtyp AS OBJECT(
num  NUMBER(6),
nom  VARCHAR2(20),
adresse  adresse_objtyp
);

CREATE TABLE client OF client_objtyp (num PRIMARY KEY)
```

1 Type abstrait de données(TAD ou type Object)

- **Modification d'un Type abstrait de données**

```
ALTER TYPE   NomType   AS OBJECT
{{ADD|MODIFY} attribute
{NomAttr [type_att]| (nomAttr type_att[,
nomAttr type_att]….)}
|DROP attribute {NomAttr | (nomAttr [, nomAttr
]….)}
```

- **Suppression d'un Type abstrait de données**

```
DROP TYPE   NomType
```

1 Type abstrait de données (Héritage)

- **Héritage entre types**

```
CREATE TYPE TPersonne  AS OBJECT (
Nump number,
nom varchar(20)
)NOT FINAL;
```

```
CREATE TYPE TConducteur UNDER  Tpersonne (
NumPermis number,
datePermis date);
```

1- TAD (Utilisation de pointeur REF)

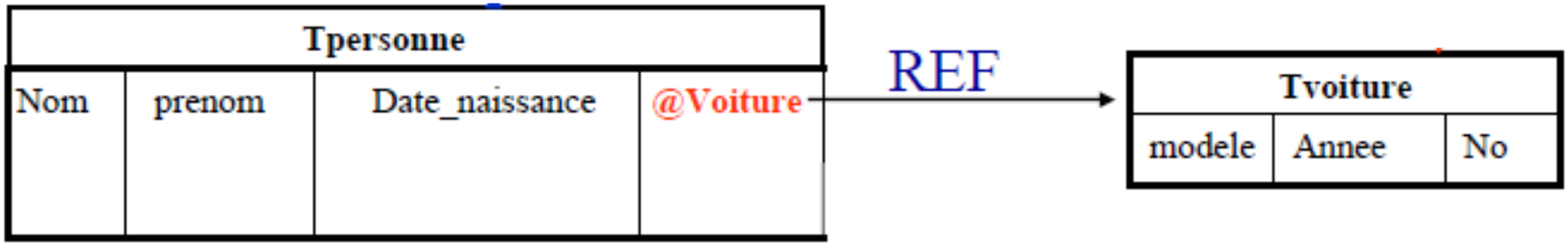
bases de données avancées

Définition

- Une définition par référence correspond à un objet de type pointeur sur l'objet lié.
- Les références peuvent être vues comme une extension de la notion de clé étrangère, et elles représentent le principal moyen pour représenter l'intégrité référentielle dans les données objet comme les types définis par l'utilisateur.
- Une référence est définie par l'utilisation du mot clé **REF** et, en interne, elle utilise l'**OID** (object Identifier) pour définir sa cible.

1 TAD (Utilisation de pointeur REF)

Exemple 1:




```
Create type Tvoiture as object (modele varchar2(15), annee date,
No integer)
/
Create or replace type Tpersonne as object
(nom varchar2(15),
 prenom varchar2(15),
 date_naissance Date,
 voiture REF Tvoiture)
/
Create table voitures of Tvoiture;
Create table personnes of Tpersonne;
```

1 TAD (Utilisation de pointeur REF)

Table voitures :

modele	annee	No
2ch	1975	128
Mégane	1998	371

Table Personnes

Nom	{Liste_prenom}	Date_naissance	@voiture
	Prenoms		
Alami	Ahmed	16-05-1963	
	Karim		
	Brahim		
Alaoui	Fatima	29-02-1944	
	Laila		

1 TAD (Utilisation de pointeur REF)

Exemple 2:

```
CREATE TYPE article_typ AS OBJECT(
num  NUMBER,
lib  VARCHAR2(10),
prxUni  NUMBER(6,2),
);
CREATE TYPE ln_commande_typ AS OBJECT(
num  NUMBER,
qtCom  NUMBER,
article  REF article_typ
);
CREATE TABLE article OF article_typ
 (CONSTRAINT pk_num PRIMARY KEY (num));

CREATE TABLE ln_commande OF ln_commande_typ (PRIMARY
KEY (num))
```

1 TAD (Utilisation de pointeur REF)

Exemple 3:

```
CREATE TYPE t_ville AS OBJECT (
nom VARCHAR(10),
population NUMBER ) ;

CREATE TABLE pays (       \\ la table est relationnelle
nom VARCHAR(30),
capitale REF t_ville,
population NUMBER ) ;

\\ Deuxième solution  en créant un autre type objet et sa table
objet :
CREATE TYPE t_pays AS OBJECT (
nom VARCHAR(30),
capitale REF t_ville,
population NUMBER );
CREATE TABLE pays OF t_pays ;
CREATE TABLE villes OF t_ville ;
```

Remarque : **Un objet tout seul n'a pas d'OID, c'est un objet d'une table objet relationnelle qui possède un OID !!!**

1 TAD (Type incomplet ou Interdépendant)

On ne peut pas créer un type qui utilise un type qui n'a pas encore été créé. on est face à ce problème dans le cas des références croisées (par exemple, un type ville qui pointe vers un type pays et un type pays qui pointe sur un type ville) ? On déclare un type incomplet (**Type Vide**) puis on le mettra à jour plus tard.

```
CREATE TYPE t_ville ; -- création du type incomplet
CREATE TYPE t_pays AS OBJECT
( … ,
 ville REF t_ville ) ;
CREATE OR REPLACE  type t_ville AS OBJECT -- modification du type
incomplet
… ,
pays REF t_pays,
… ) ;
```

2 Les collections imbriquées
a- Tableau (Varray)

2-a Tableau prédimentionné (Varray)

Permet d'avoir des attributs multivalués en définissant une collection ordonnée et limitée (<taille>) d'éléments de même type.

```
CREATE TYPE <nomType> AS VARRAY(<taille>) OF <type>;
```

Type : Le type peut être n'importe quel type, prédéfini de SQL (VarChar, Date, Number…) ou défini par l'utilisateur TAD.

Exemple: plusieurs prénoms pour une personne

nom	{liste_prenom}	Date_naissance	voitures		
	prenoms		modele	annee	no
Alami	Ahmed	16/05/1963	2ch	1975	128
	Karim				
Alaoui	Fatima	29/02/1944	Twingo	1999	17
	Laila				

2 -a Tableau prédimentionné (Varray)

```
CREATE TYPE liste_prenom as Varray(10) of varchar2(15);

CREATE OR REPLACE TYPE Tpersonne AS OBJECT
(nom varchar2(15),
 Date_n Date,
 prenoms liste_prenom,
 voiture Tvoiture);

 CREATE TABLE personnes of Tpersonne;
```

- Varray(10) : Varray est le mot-clé pour définir une collection et 10 le nombre maximum d'éléments de cette liste.

Le type des éléments de la collection est spécifié après le mot-clé 'of

2 Les collections imbriquées

b- Table (as Table of)

2-b Table imbriquée (as table of)

a-Définition Permet d'avoir des attributs multivalués de type table, ce type est utilisé lorsqu'il n'y pas de nombre maximum de valeurs, l'ordre n'est pas important;

b-Syntaxe : **CREATE TYPE nom-type AS TABLE OF nom-type2**

C- Exemple:

- Déclarations du type de la table imbriquée (AS TABLE OF)

```
CREATE TYPE t_employe AS OBJECT //type pour une table imbriquée
( num_insee VARCHAR(20),
  nom VARCHAR(30),
  age NUMBERage NUMBER
);

CREATE TYPE t_employes AS TABLE OF t_employe ; // t_employe est le
type des éléments de la table
```

2-b Table imbriquée (as table of)

bases de données avancées

- Création de la table imbriquée (Nested Table)

Une nested table est une table à l'intérieur d'une autre table, elle est particulièrement utilisée dans la modélisation des relations 1-n et des liens identifiants

1 - table relationnelle standard

-- tab_emp est le nom physique de la table imbriquée (ne sert jamais dans les requêtes)

```
CREATE TABLE departements
( num_dep NUMBER,
  budget NUMBER,
  employes t_employes)
  NESTED TABLE employes STORE AS tab_emp ;
```

2- table relationnelle Objet

```
CREATE TYPE t_departement AS OBJECT
( num_dep NUMBER,
  budget NUMBER,
  employes t_employes);

CREATE TABLE departements OF t_departement NESTED TABLE employes
STORE AS tab_emp ;
```

2-b Table imbriquée (as table of)

Exemple: Table des references

```
Create type T_Pays;
Create type T_Ville as object (
Nom varchar2(30),
Pays ref  T_Pays,
Population number
) ;
Create type EnsVille as table of ref  T_Villes ;
Create or replace type T_Pays as Object (
Nom varchar2(30),
Capitale ref  T_Ville,
VillesPrincipales EnsVilles,
Population number,
) ;
Create table LesPays of T_Pays(
Nested table villesPrincipales store as tabvilles)
Create table Lesville of T_Ville ;
```

2-b Table imbriquée (as table of)

Imbrication multiniveau

Il est possible d'imbriquer les types ensemblistes de manière à définir :

- **des VARRAY de VARRAY**

```
CREATE TYPE V1 AS VARRAY(10) OF tObj;
CREATE TYPE V2 AS VARRAY(5) OF V1;
```

- **des VARRAY de TABLE**

```
CREATE TYPE T1 AS TABLE OF tObj;
CREATE TYPE V2 AS VARRAY(5) OF T1;
```

- **des TABLE de VARRAY**

```
CREATE TYPE V1 AS VARRAY(10) OF tObj;
CREATE TYPE T2 AS TABLE OF V1;
```

- **des TABLE de TABLE**

```
CREATE TYPE T1 AS TABLE OF tObj;
CREATE TYPE T2 AS TABLE OF T1;
```

Modélisation avec un Exemple
- Du Relationnel vers RO-

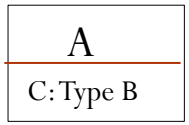
SQL3: Modélisation/Règles de transformation

Il ne s'agit pas de reproduire le modèle relationnel mais de l'optimiser en dénormalisant.
La solution est de Dénormaliser en utilisant soit:

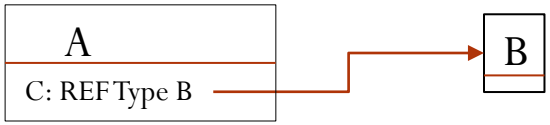
- L'imbrication
- La référence
- Les deux

- **Cas de 1-1**

- Création d'une seule table dans laquelle un de ses attribut est de type de l'autre relation



- Création de deux tables avec un attribut de type REF vers l'autre Table

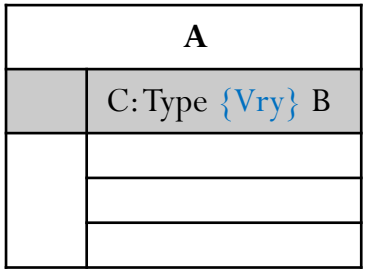


SQL3: Modélisation/Règles de transformation

bases de données avancées

- Cas de 1-N

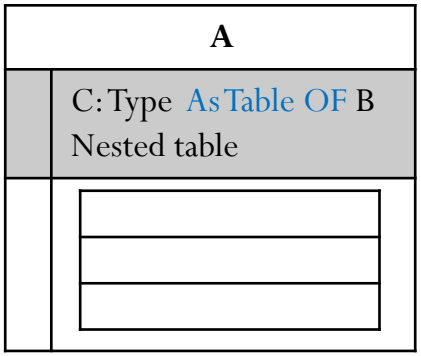
A. Si le n est limité: Type tableau (VARRAY) dans la dénitrions de la table père. (Création d'une seule table)



B. Si le n n'est pas limité:

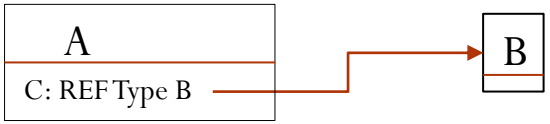
- Utiliser une table imbriquée (NESTED TABLE)

(Création d'une seule table) Table Père qui imbrique la relation fils



- Utiliser les références

(Comme dans le relationnel Création de deux table)

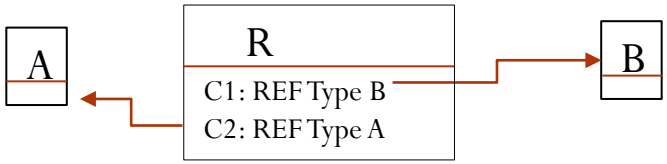


SQL3: Modélisation/Règles de transformation

- Cas de N-N

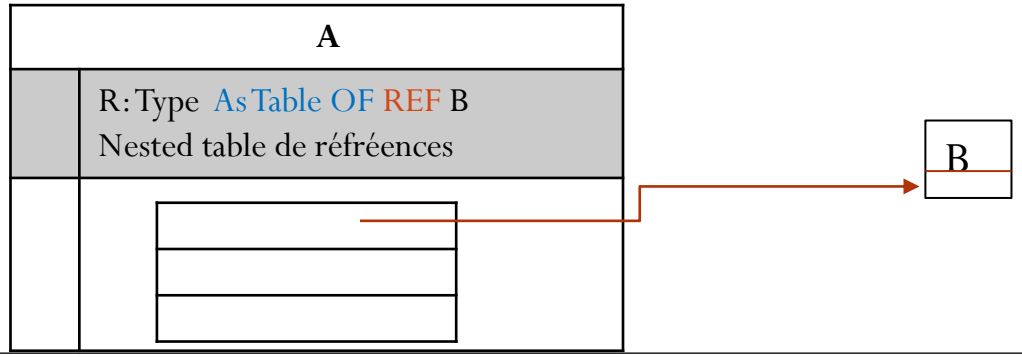
A. Utilisation des références (pas d'imbrications)

Création de 3 Tables. Comme dans le relationnel, une table association est rajoutée avec des références vers les deux autres tables



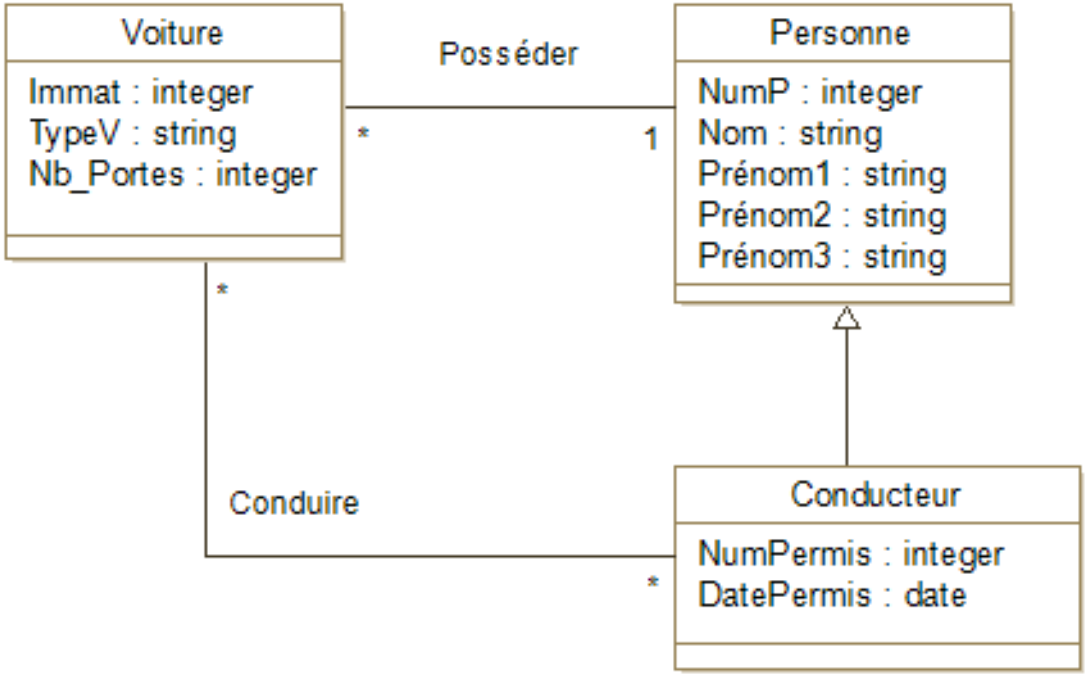
B. Utilisation des imbrications: Table imbriquée de références

- l'imbrication ne doit concerner que la relation. il faudra imbriquer la table Relation (Nested TABLE) dans l'une des deux tables soit la table 1 ou la table 2.
- La table choisie pour l'imbrication est selon l'utilisation de l'application à développer. La table la plus utilisée sera choisie pour l'imbrication



SQL3: Modélisation/Création des types

<u>Exemple</u>: Transformation du modèle relationnel



Personne (<u>NumP</u>, Nom, Prénom1, Prénom2, Prénom3);
Conducteur (<u>NumP</u>*, NumPermis, DatePermis)
Voiture (<u>Immat</u>, TypeV, Nb_portes, NumP***)**
Conduire (LeConducteur***,** LaVoiture***)**

SQL3: Modélisation/Création des types

Sol 1: Solution avec références (<u>création de 4 tables</u>) proche du modèle relationnel

Tvoiture/**Voiture**

Imat	TypeV	Nb_Portes	Propriétaire Ref TPersonne

Tpersonne/**Personne**

Nump	Nom	Prenoms {Vry}

3 maximum

TConduire/**Conduire**

LeConducteur Ref TConducteur	LaVoiture Ref TVoiture

TConducteur/**Conducteur**

Nump	Nom	Prenom	NumPermis	DatePermis

SQL3: Modélisation/Création des types

Sol 1: Solution avec références (<u>création de 4 tables</u>) proche du modèle relationnel

Tvoiture/**Voiture**

Imat	TypeV	Nb_Portes	Propriétaire Ref TPersonne
13-119-456	citadine	5	select REF(P) from Personne P where P.NUMP=1
13-120-358	crossover	4	

Tpersonne/**Personne**

Nump	Nom	Prenoms {Vry}
1	Mahmoud	Tarik
		ismail
		NULL
2	ARABI	Mohammed
		NULL
		NULL

3 maximum

TConduire/**Conduire**

LeConducteur Ref TConducteur	LaVoiture Ref TVoiture

TConducteur/**Conducteur**

Nump	Nom	Prenom	NumPermis	DatePermis
1	mahmoud	Tarik	56799	15/2/2010
		ismail		
		NULL		

SQL3: Création des types

```
CREATE TYPE TListePrenoms as
varray(3) OF varchar(15);

CREATE TYPE TPersonne  as
object (
Nump number,
nom varchar(20),
prenom TListePrenoms
)not final;

CREATE TYPE TConducteur
under  Tpersonne (
NumPermis number,
datePermis date) ;
```

```
CREATE TYPE TVoiture as object (
  Immat varchar(20),
  TypeV varchar(20),
  Nb_Portes number,
  Proprietaire REF TPersonne
) ;
CREATE TYPE TConduire as object (
LeConducteur REF Tconducteur,
LAVoiture REF TVoiture
)

CREATE TABLE Personne of TPersonne ;
CREATE TABLE Conducteur of
Tconducteur ;
CREATE TABLE Voiture of TVoiture;
CREATE TABLE Conduire of TConduire;
```

SQL3: Modélisation/Création des types

Sol 2: Solution avec références (création de trois tables) centrée sur Voitures

Tvoiture/**Voiture**

Imat	TypeV	Nb_Portes	Propriétaire	LesConducteurs {Conducteur_nestedTab}
				Ref Conducteur

Tpersonne/**Personne**

Nump	Nom	Prenom {Vry}

3 maximum

Plusieurs lignes,
Ce n'est pas limité

TConducteur/**Conducteur**

Nump	Nom	Prenom	NumPermis	DatePermis

SQL3: Création des types

```
CREATE TYPE TListePrenoms AS
VARRAY(3) OF varchar(15);

CREATE TYPE TPersonne  AS
OBJECT (
Nump number,
nom varchar(20),
prenom TListePrenoms
)NOT FINAL;

CREATE TYPE TConducteur
UNDER  Tpersonne (
NumPermis number,
datePermis date) ;

CREATE TYPE TListeConducteurs AS
TABLE OF REF TConducteur;
```

```
CREATE TYPE TVoiture AS OBJECT (
  Immat varchar(20),
  TypeV varchar(20),
  Nb_Portes number,
  Proprietaire REF TPersonne,
  LesConducteurs TListeConducteurs) ;

CREATE TABLE Personne OF TPersonne ;

CREATE TABLE Conducteur OF
Tconducteur ;

CREATE TABLE Voiture OF TVoiture
NESTED TABLE LesConducteurs STORE AS
TabConducteurs;
```

SQL3: Modélisation/Création des types

bases de données avancées

Sol 3: Une seule Table (Voiture) , solution sans référence

Tvoiture/**Voiture**

Imat	TypeV	Nb_Portes	Propriétaire			LesConducteurs {Conducteur_nestedTab}				
			Nump	Nom	Prenom {vry}	Nump	Nom	Prenom	NumPermis	DatePermis
13-120-358	Crossover	4	2	ARABI	Mohammed	1	mahmoud	Tarik	56799	15/2/2010
					NULL			ismail		
					NULL			NULL		
						2	ARABI	Mohammed	890543	10/13/2015
								NULL		
								NULL		
						3	Senouci	Leila	210543	11/04/2011
								souad		
								NULL		

Tpersonne

Nump	Nom	Prenom {vry}

TConducteur

Nump	Nom	Prenom	NumPermis	DatePermis

Plusieurs lignes,
Ce n'est pas limité, on peut ajouter d'autres conducteurs

SQL3: Création des types

```
CREATE TYPE TListePrenoms AS
VARRAY(3) OF varchar(15);

CREATE TYPE TPersonne  AS
OBJECT (
Nump number,
nom varchar(20),
prenom TListePrenoms
)NOT FINAL;

CREATE TYPE TConducteur UNDER
Tpersonne (
NumPermis number,
datePermis date);

CREATE TYPE TListeConducteurs
AS TABLE  OF  TConducteur;
```

```
CREATE TYPE TVoiture AS OBJECT (
  Immat varchar(20),
  TypeV varchar(20),
  Nb_Portes number,
  Proprietaire  TPersonne,
  LesConducteurs
TListeConducteurs);

CREATE TABLE Voiture OF TVoiture
NESTED TABLE LesConducteurs STORE
AS TabConducteurs;
```

SQL3: Modélisation/Création des types

Sol 4: Solution avec références (création de trois tables) centrée sur Conducteur

Tvoiture/**Voiture**

Imat	TypeV	Nb_Portes	Propriétaire

Tpersonne/**Personne**

Nump	Nom	Prenom {Vry}

3 maximum

TConducteur/**Conducteur**

Nump	Nom	Prenom	NumPermis	DatePermis	LesVoitures {Voiture Nested tab}
					Ref Voiture

SQL3: Création des types

<u>A faire</u> (At home):

Donner la syntaxe de la solution 3 et proposer une solution avec seulement deux tables: Voitures et conducteurs